Otimização da Eficiência de Rede Assistida por Aprendizado Federado Usando IA Generativa e Imagens

Gian Pedro Rodrigues*, Herman L. dos Santos*,
*Departamento Academico de Engenharia Elétrica, Universidade Tecnológica Federal do Paraná, Cornélio Procópio, Brasil
E-mail: hermansantos@utfpr.edu.br, gian.2000@alunos.utfpr.edu.br

***Resumo*—Este trabalho apresenta um framework containerizado para investigar a comunicação semântica de imagens utilizando aprendizado federado e IA generativa. O sistema emprega tanto um autoencoder convolucional determinístico quanto um *Variational Autoencoder* (VAE) — modelo genuinamente generativo — treinados de forma distribuída entre múltiplos dispositivos simulados por contêineres Docker. Ambos os modelos comprimem imagens do dataset MNIST de 784 pixels para apenas 32 valores no espaço latente (redução de 96%), sendo que o VAE aprende uma distribuição contínua no espaço latente capaz de gerar novas imagens sem precedentes nos dados de treinamento. O treinamento federado utiliza o algoritmo FedAvg com compressão Top-K dos pesos do modelo, enquanto um mecanismo de injeção de caos baseado em `tc netem` avalia a resiliência sob condições adversas de rede. Adicionalmente, o sistema incorpora simulação de canal AWGN no espaço latente, aproximando-se de um esquema de codificação conjunta fonte-canal (JSCC). Os resultados demonstram: (i) a viabilidade da comunicação semântica generativa via FL; (ii) a superioridade do treinamento centralizado sobre o federado em qualidade pura, evidenciando o trade-off com privacidade; (iii) a robustez do framework sob quatro cenários de caos; e (iv) a capacidade de completação de imagens parciais e geração de novas amostras pelo VAE.**

***Palavras-chave*—Aprendizado Federado, Comunicação Semântica, IA Generativa, Variational Autoencoder, Docker, Injeção de Caos, JSCC, SSIM**

I. Introdução

O crescimento exponencial de dispositivos conectados na Internet das Coisas (IoT) e o avanço das redes sem fio 5G/6G têm gerado desafios significativos em termos de largura de banda, latência e privacidade [**?**]. Sistemas tradicionais de comunicação, fundamentados na teoria de Shannon [**?**], focam em transmitir bits com máxima fidelidade, ignorando o *significado* da informação. Por exemplo, ao transmitir uma imagem de um dígito manuscrito, o sistema clássico precisa enviar todos os 784 pixels, mesmo que a informação semântica possa ser representada de forma muito mais compacta.

Nesse contexto, a **comunicação semântica** surge como paradigma promissor: o transmissor extrai a essência da informação e envia uma representação compacta ao receptor, que reconstrói a mensagem original [**?**], [**?**]. Recentemente, modelos de **IA generativa** — como Variational Autoencoder (VAE)s, GANs e modelos de difusão — têm sido propostos como codecs semânticos avançados, capazes não apenas de comprimir, mas de *gerar* conteúdo semanticamente coerente no receptor [**?**], [**?**]. Paralelamente, o **Federated Learning (FL)** consolidou-se como abordagem para treinar modelos de forma descentralizada, onde cada dispositivo treina localmente e compartilha apenas os parâmetros do modelo — nunca os dados brutos [**?**].

A combinação dessas tecnologias permite que dispositivos distribuídos colaborem no treinamento de um codec semântico generativo sem compartilhar dados privados. Este trabalho propõe um **testbed prático e reprodutível** que integra essas abordagens, utilizando contêineres Docker para simular um cenário distribuído. Dois modelos são avaliados: um **autoencoder convolucional** determinístico e um **Variational Autoencoder (VAE)** — modelo genuinamente generativo [**?**] — que aprende uma distribuição no espaço latente, permitindo geração de novas amostras e comunicação semântica mais robusta. Ambos comprimem imagens MNIST para um vetor de 32 dimensões (compressão de 96%), e permitem testar a **completação de imagens parciais**. Adicionalmente, o sistema incorpora simulação de canal Additive White Gaussian Noise (AWGN) no espaço latente, aproximando o conceito de codificação conjunta fonte-canal (Joint Source-Channel Coding (JSCC)) [**?**].

As principais contribuições deste trabalho são:

- Um framework containerizado e reprodutível para treinamento federado e comunicação semântica generativa de imagens, orquestrado via Docker Compose;
- Um VAE como codec semântico generativo, capaz de comprimir, reconstruir e *gerar* novas imagens a partir de uma distribuição aprendida no espaço latente [**?**];
- Comparação sistemática entre AE determinístico e VAE generativo, e entre treinamento centralizado e federado, evidenciando trade-offs de qualidade vs. privacidade;
- Simulação de canal AWGN no espaço latente, aproximando o conceito de JSCC;
- Compressão Top-K dos pesos do modelo (60% de esparsidade) para reduzir o custo de comunicação durante o treinamento federado;
- Avaliação com métricas MSE, PSNR e Structural Similarity Index (SSIM) [**?**], alinhada ao estado da arte em comunicação semântica;
- Um mecanismo configurável de injeção de caos baseado em `tc netem` para avaliar a robustez do sistema sob condições adversas.

O restante do artigo está organizado da seguinte forma: a Seção II apresenta os trabalhos relacionados; a Seção III descreve a arquitetura proposta em detalhes; a Seção IV apresenta os resultados experimentais; e a Seção **??** conclui o trabalho com discussões e direções futuras.

II. Trabalhos Relacionados

A. Aprendizado Federado

O FL foi formalizado por McMahan et al. [**?**], que propuseram o FedAvg para agregar modelos treinados localmente. Cada dispositivo treina uma cópia local e envia apenas os pesos ao servidor, que calcula a média dos parâmetros. Diversas extensões abordaram desafios como dados heterogêneos (non-IID) [**?**], compressão de gradientes [**?**] e robustez contra falhas [**?**]. Li et al. [**?**] e Kairouz et al. [**?**] compilaram os principais avanços e problemas em aberto, destacando a necessidade de compressão eficiente para cenários com largura de banda limitada.

B. Comunicação Semântica

A comunicação semântica, inspirada nos três níveis de comunicação de Shannon e Weaver [**?**], ganhou impulso com deep learning. Yang et al. [**?**] revisaram fundamentos e desafios, apontando autoencoders como arquitetura promissora. O DeepSC [**?**] utilizou redes neurais para codificação semântica de texto. Bourtsoulatze et al. [**?**] propuseram autoencoders para JSCC de imagens, demonstrando que redes neurais podem superar esquemas tradicionais em baixa relação sinal-ruído. Weng e Qin [**?**] estenderam a abordagem para sinais de fala. Luo et al. [**?**] destacaram a necessidade de testbeds práticos, e Kurka e Gündüz [**?**] propuseram o DeepJSCC-f com feedback de canal.

C. IA Generativa para Comunicação Semântica

Recentemente, modelos generativos emergiram como evolução natural dos codecs semânticos baseados em autoencoders determinísticos. Grassucci et al. [**?**] apresentaram uma taxonomia abrangente dos modelos generativos — VAEs, GANs e modelos de difusão — aplicados à comunicação semântica, destacando que modelos generativos permitem não apenas comprimir, mas *gerar* conteúdo no receptor a partir de uma descrição semântica compacta. Du et al. [**?**] propuseram o uso de modelos de difusão para comunicação semântica sem necessidade de treinamento conjunto transmissor-receptor.

No contexto de redes wireless, Du et al. [**?**] introduziram o GenAINet, um framework que utiliza IA generativa para inteligência coletiva em redes sem fio, propondo mecanismos de transferência de conhecimento e raciocínio no edge. Xia et al. [**?**] formalizaram a *camada generativa de rede*, posicionando o modelo generativo como componente integral do protocolo de comunicação, capaz de adaptar-se dinamicamente às condições do canal.

O VAE, proposto por Kingma e Welling [**?**], é particularmente adequado para comunicação semântica por aprender uma distribuição contínua no espaço latente $p(z) = \mathcal{N}(0, I)$, o que confere regularidade ao espaço de representações e permite geração de novas amostras. Higgins et al. [**?**] introduziram o β-VAE, onde o hiperparâmetro β controla o trade-off entre qualidade de reconstrução e regularidade do espaço latente — conceito diretamente relevante para comunicação, onde o espaço latente funciona como canal de transmissão.

Neste trabalho, posicionamos nosso framework no contexto dessas referências: implementamos e comparamos um AE determinístico com um VAE generativo, ambos treinados via FL, com simulação de canal AWGN — abordando lacunas identificadas na literatura recente.

D. Testbeds e Engenharia de Caos

A maioria dos trabalhos na área utiliza simulações idealizadas, sem imperfeições reais de rede. O conceito de **engenharia de caos**, popularizado pela Netflix [**?**], consiste em introduzir falhas intencionais para avaliar a resiliência de um sistema. A ferramenta `tc netem` do Linux permite simular perda de pacotes, latência e corrupção em interfaces de rede reais. Neste trabalho, aplicamos esse conceito ao treinamento federado, executando comunicação real entre processos isolados com injeção de imperfeições configuráveis em tempo real.

III. Sistema Proposto

Nesta seção, descrevemos cada componente do framework proposto.

A. Visão Geral da Arquitetura

O sistema é composto por cinco contêineres Docker orquestrados via Docker Compose. Cada contêiner simula um componente de um sistema distribuído de comunicação, conforme descrito na Tabela I.

Tabela I
Contêineres do sistema e suas funções

Contêiner	Função
`fl-server`	Servidor Flask que recebe pesos dos clientes, executa agregação FedAvg e disponibiliza o modelo global via API REST
`client-full`	Cliente estável que treina com todos os dígitos (IID) e envia pesos completos (sem compressão)
`client-noisy`	Cliente instável que aplica compressão Top-K (60% esparsidade) antes de enviar os pesos
`client-noniid`	Cliente com dados heterogêneos (Non-IID), treinando apenas com os dígitos 0–3, enviando pesos completos
`dashboard`	Painel Streamlit para visualização em tempo real, testes interativos de comunicação semântica e controle do caos
`chaos-injector`	Contêiner Alpine que aplica regras `tc netem` na interface de rede do cliente instável

Os contêineres compartilham a mesma imagem Docker (Python 3.9 + PyTorch) e comunicam-se via rede bridge `fl-network`, usando nomes de serviços como hostnames. O injetor de caos opera no namespace de rede do cliente instável

(via `network_mode: service:client-noisy`), permitindo que regras `tc netem` afetem diretamente o tráfego desse contêiner.

A inclusão de três clientes com perfis distintos permite uma avaliação mais realista do treinamento federado: o `client-full` serve como baseline estável; o `client-noisy` testa a resiliência à compressão e caos de rede; e o `client-noniid` avalia o impacto de dados heterogêneos (Non-IID) na convergência do modelo global, um dos principais desafios em FL [**?**].

B. Comunicação entre Contêineres

A comunicação é realizada via **API REST** (HTTP). O servidor Flask expõe quatro endpoints: `POST /upload` para receber pesos treinados; `GET /global_model` para distribuir o modelo global; `POST /reconstruct` para decodificar um vetor latente em imagem; e `POST /complete` para reconstruir uma imagem parcial. Essa arquitetura é extensível — novos clientes são adicionados configurando um contêiner que aponte para o servidor.

C. Autoencoder Convolucional

O modelo central do sistema é um autoencoder convolucional projetado para comprimir e reconstruir imagens MNIST ($1\times28\times28$ pixels). O autoencoder é composto por duas partes simétricas: o encoder (compressor) e o decoder (reconstrutor).

O encoder comprime a imagem (28×28 pixels) para um vetor latente de 32 dimensões através de duas camadas convolucionais com ReLU e max-pooling, seguidas de duas camadas totalmente conectadas ($3136\to256\to32$). O decoder realiza o caminho inverso, expandindo o vetor latente via camadas lineares e convolução transposta com ativação Sigmoid final. A Tabela II detalha a arquitetura completa.

Tabela II
ARQUITETURA DETALHADA DO AUTOENCODER CONVOLUCIONAL

Camada	**Tipo**	**Saída**
	Encoder	
Entrada	Imagem	$1\times28\times28$
Conv2d + ReLU	32 filtros, 3×3	$32\times28\times28$
MaxPool2d	2×2	$32\times14\times14$
Conv2d + ReLU	64 filtros, 3×3	$64\times14\times14$
MaxPool2d	2×2	$64\times7\times7$
Flatten + Linear	$3136 \to 256$	256
Linear	$256 \to 32$	**32**
	Decoder	
Linear	$32 \to 256$	256
Linear	$256 \to 3136$	3136
Reshape	—	$64\times7\times7$
ConvTranspose2d + ReLU	32 filtros	$32\times14\times14$
ConvTranspose2d + Sigmoid	1 filtro	$1\times28\times28$

A função de perda utilizada para treinamento é o Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \tag{1}$$

onde $\mathbf{x}_i$ é a imagem original e $\hat{\mathbf{x}}_i$ é a imagem reconstruída pelo decoder. Essa função penaliza diferenças pixel a pixel entre a entrada e a saída, incentivando o modelo a preservar os detalhes visuais.

A taxa de compressão alcançada pelo codec semântico é:

$$R_c = 1 - \frac{D_{\text{latente}}}{D_{\text{original}}} = 1 - \frac{32}{784} \approx 95{,}9\% \tag{2}$$

Isso significa que mais de 95% dos dados podem ser descartados durante a transmissão, enviando-se apenas 32 números reais em vez de 784 pixels.

D. Variational Autoencoder (VAE) como Codec Generativo

Embora o autoencoder determinístico comprima eficientemente, ele não é um modelo generativo — não permite amostrar novas imagens do espaço latente. Para incorporar IA generativa ao framework, implementamos um VAE [**?**] como alternativa ao AE.

O VAE utiliza o mesmo backbone convolucional (encoder e decoder), com a diferença fundamental de que a última camada do encoder é bifurcada em duas cabeças: $\boldsymbol{\mu}$ (média) e $\log \boldsymbol{\sigma}^2$ (log-variância), produzindo uma distribuição posterior $q(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\mathbf{I})$. A amostragem utiliza o *reparameterization trick* [**?**]:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{3}$$

A função de perda do VAE combina reconstrução e regularização:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathcal{L}_{\text{MSE}}}_{\text{reconstrução}} + \beta \cdot \underbrace{D_{\text{KL}}\big(q(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\big)}_{\text{regularização}} \tag{4}$$

onde a divergência Kullback-Leibler (KL) tem forma analítica:

$$D_{\text{KL}} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \tag{5}$$

O hiperparâmetro β (conceito de β-VAE [**?**]) controla o trade-off entre fidelidade de reconstrução e regularidade do espaço latente: $\beta > 1$ favorece um espaço latente mais organizado (útil para geração), enquanto $\beta < 1$ prioriza reconstrução. Neste trabalho, utilizamos $\beta = 1$ (VAE padrão).

A principal vantagem do VAE para comunicação semântica é que o espaço latente regularizado $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ permite: (i) *geração* — novas imagens são criadas amostrando $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ e decodificando; (ii) *robustez ao canal* — a regularidade do espaço latente torna a representação mais tolerante a perturbações; e (iii) *interpolação* — transições suaves entre representações semânticas no espaço latente. A Tabela III resume as diferenças arquiteturais.

Tabela III
COMPARAÇÃO ARQUITETURAL AE VS. VAE

Característica	AE	VAE
Encoder saída	Vetor $\mathbf{z} \in \mathbb{R}^{32}$	$\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2 \in \mathbb{R}^{32}$
Espaço latente	Determinístico	Probabilístico
Loss	MSE	MSE + $\beta\cdot$ KL
Generativo	Não	Sim
Parâmetros	1.663.265	1.671.489

E. Modelo de Canal AWGN no Espaço Latente

Para aproximar o cenário real de comunicação wireless, incorporamos um modelo de canal AWGN diretamente no espaço latente, seguindo a abordagem de JSCC proposta por Bourtsoulatze et al. [**?**]. Durante o treinamento, o vetor latente é corrompido por ruído gaussiano antes da decodificação:

$$\hat{\mathbf{z}} = \mathbf{z} + \mathbf{n}, \quad \mathbf{n} \sim \mathcal{N}\left(\mathbf{0}, \sigma_n^2 \mathbf{I}\right) \tag{6}$$

onde $\sigma_n = 1/\sqrt{10^{\text{SNR}_{\text{dB}}/10}}$. Ao treinar com o canal AWGN ativo, o modelo aprende representações latentes mais robustas a perturbações, efetivamente realizando codificação conjunta fonte-canal: o encoder atua simultaneamente como compressor semântico e codificador de canal, enquanto o decoder realiza decodificação conjunta.

F. Métricas de Avaliação

Além do MSE (Eq. 1) e do Peak Signal-to-Noise Ratio (PSNR), este trabalho emprega o SSIM [**?**], métrica padrão na literatura de comunicação semântica que avalia a similaridade estrutural entre imagens:

$$\text{SSIM}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)} \tag{7}$$

O SSIM captura similaridade de luminância, contraste e estrutura, com valor 1 indicando imagens idênticas. Diferentemente do MSE (puramente pixel a pixel), o SSIM correlaciona-se melhor com a percepção humana de qualidade visual.

G. Protocolo de Treinamento Federado

O treinamento é realizado de forma distribuída utilizando o algoritmo FedAvg [**?**]. O Algoritmo 1 descreve o protocolo completo.

A cada rodada, os clientes treinam localmente por 5 épocas (batch=32, Adam, lr=0,001) e enviam os pesos ao servidor, que os agrega pela média com $K = 3$ clientes. O `client-noisy` aplica **compressão Top-K**: apenas 40% dos pesos são enviados, e os restantes são reconstruídos no servidor por *imputação pela média*. O `client-noniid` treina apenas com dígitos 0–3, simulando heterogeneidade de dados (non-IID) [**?**].

H. Comunicação Semântica de Imagens

O sistema implementa dois modos de teste de comunicação semântica, ambos acessíveis interativamente pelo painel de visualização:

Algoritmo 1 Treinamento Federado com FedAvg e Top-K

Entrada: Servidor S, clientes $\{C_1, C_2, C_3\}$, rodadas T
Saída: Modelo global $\mathbf{w}^{(T)}$

```
Inicializar w^(0) aleatoriamente no servidor
para t = 0 até T − 1 faça
    S disponibiliza w^(t)
    para cada cliente k ∈ {1, 2, 3} em paralelo faça
        C_k obtém w^(t) do servidor
        C_k treina localmente por 5 épocas
        se C_k é cliente instável então
            Aplica compressão Top-K (40% dos pesos)
        fim se
        se C_k é cliente Non-IID então
            Treina apenas com subconjunto de classes (dígitos 0–3)
        fim se
        C_k envia w_k^(t+1) ao servidor
    fim para
    S agrega: w^(t+1) = (1/K) Σ_{k=1}^{K} w_k^(t+1)
fim para
```

Modo Compressão: O transmissor codifica a imagem via encoder, obtendo um vetor latente de 32 dimensões que é transmitido ao receptor. Este utiliza o decoder para reconstruir a imagem original, avaliando a qualidade em termos de MSE, PSNR e SSIM.

Modo Completação: O transmissor envia apenas uma fração da imagem original, com três tipos de mascaramento: metade inferior zerada (transmissão interrompida), pixels aleatórios zerados (perda de dados) ou metade direita zerada (informação espacial parcial). O modelo reconstrói a imagem completa, incluindo a parte mascarada, demonstrando que aprendeu uma representação semântica capaz de inferir conteúdo visual faltante.

Modo Geração (apenas VAE): Ao utilizar o VAE, o sistema permite gerar novas imagens sem precedentes nos dados de treinamento, amostrando $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ e decodificando. Essa capacidade demonstra que o modelo é genuinamente generativo — diferentemente do AE determinístico, que apenas reconstrói exemplos observados.

I. Injeção de Caos

O mecanismo de injeção de caos utiliza a ferramenta `tc netem` do kernel Linux para alterar dinamicamente as condições de rede do cliente instável. O contêiner `chaos-injector` executa um loop shell que lê continuamente um arquivo de configuração (`chaos_config.txt`) e aplica as regras correspondentes na interface de rede `eth0`.

Quatro parâmetros são configuráveis em tempo real: **perda de pacotes** (0–5%), **latência** (0–2000 ms), **corrupção** (0–2%) e **duplicação** (0–5%). Essas condições afetam exclusivamente o cliente instável, permitindo comparação direta com os clientes estável e Non-IID sob condições ideais.

J. Painel de Visualização

O painel Streamlit oferece seis abas para monitoramento em tempo real: *Arquitetura* (topologia e perfis dos clientes), *Comunicação* (teste de reconstrução semântica com métricas), *Completação* (teste com imagens parciais), *Terminais* (logs do servidor e clientes), *Métricas* (curvas de convergência por rodada) e *Experimentos* (comparação entre cenários de caos). A barra lateral controla o treinamento (play/pause) e os parâmetros de caos.

IV. Resultados Experimentais

Nesta seção, apresentamos os resultados da avaliação experimental do framework proposto. Os experimentos abrangem a convergência do treinamento federado, a qualidade da reconstrução semântica, os testes de completação de imagens parciais e o impacto da injeção de caos.

A. Configuração Experimental

Os experimentos foram conduzidos utilizando Docker Desktop em ambiente local com Windows. O dataset utilizado é o MNIST [**?**], um benchmark amplamente adotado na comunidade de aprendizado de máquina, contendo 60.000 imagens de treinamento e 10.000 de teste. Cada imagem representa um dígito manuscrito (0–9) em escala de cinza com resolução 28×28 pixels. A Tabela IV apresenta os principais parâmetros do sistema.

Tabela IV
Parâmetros experimentais

Parâmetro	**Valor**
Dataset	MNIST (28×28, 1 canal)
Imagens de treinamento	60.000
Imagens de teste	10.000
Dimensão latente	32
Pixels por imagem	784
Taxa de compressão	95,9%
Fator de compressão	24,5×
Épocas locais por rodada	5
Tamanho do batch	32 imagens
Otimizador	Adam (lr = 0,001)
Esparsidade Top-K (noisy)	60%
Número de clientes	3
Cliente Non-IID	Dígitos 0–3
Modelos avaliados	AE e VAE ($\beta = 1$)
Canal AWGN	SNR = $\{5, 10, 15, 20\}$ dB
Métricas	MSE, PSNR, SSIM
Framework ML	PyTorch 2.x
Contêineres	6 (Docker Compose)

B. Convergência do Treinamento Federado

A primeira análise diz respeito à convergência do treinamento federado ao longo das rodadas. A Figura 1 apresenta a evolução da loss (MSE) para os três clientes durante o treinamento.

Os resultados mostram que os três clientes apresentam convergência progressiva ao longo das rodadas. O `client-full`, com dados IID e pesos completos, converge de forma mais rápida e estável. O `client-noisy` apresenta

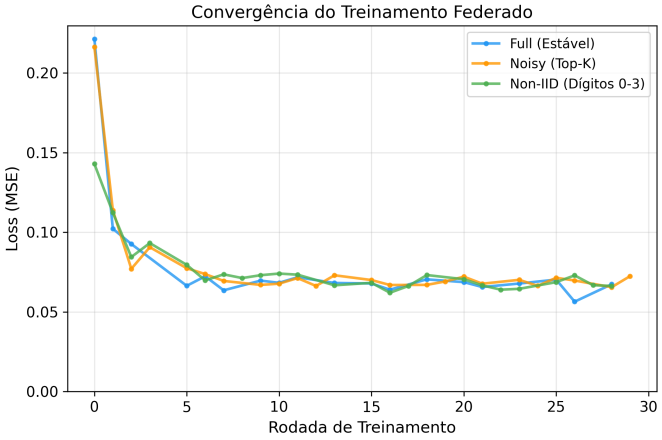

Figura 1. Evolução da loss (MSE) ao longo das rodadas de treinamento federado para os três clientes: `client-full` (azul), `client-noisy` (laranja) e `client-noniid` (verde).

convergência mais ruidosa devido à compressão Top-K e às condições adversas de rede. O `client-noniid` demonstra uma tendência de convergência intermediária: apesar de enviar pesos completos, a distribuição enviesada de dados (apenas dígitos 0–3) resulta em gradientes que não representam a distribuição completa dos dados, afetando a contribuição para o modelo global. A agregação FedAvg, ao calcular a média dos pesos dos três clientes, mitiga parcialmente esses efeitos, resultando em um modelo global funcional mesmo com fontes heterogêneas de dados e compressão agressiva.

C. Qualidade da Reconstrução Semântica

Para avaliar a qualidade do codec semântico, comparamos imagens originais do dataset de teste com suas reconstruções. A qualidade é medida pelo **MSE** (Mean Squared Error), onde valores menores indicam maior fidelidade, e pelo **PSNR** (Peak Signal-to-Noise Ratio), expresso em dB, onde valores maiores indicam melhor reconstrução.

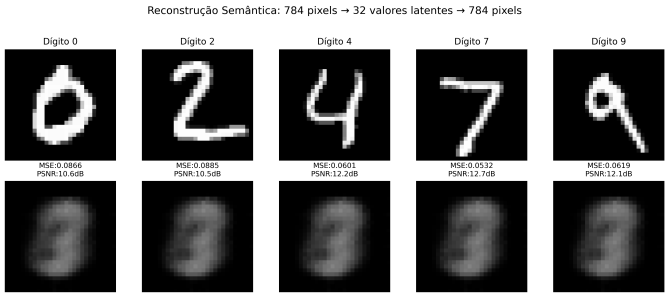

Figura 2. Exemplos de imagens originais (topo) e reconstruções semânticas (base) geradas pelo autoencoder treinado federadamente. Abaixo de cada reconstrução, MSE e PSNR individuais.

A Tabela V confirma que o autoencoder preserva a informação semântica com MSE médio de 0,073 e PSNR médio

Tabela V
MÉTRICAS DE RECONSTRUÇÃO SEMÂNTICA POR DÍGITO.

Dígito	MSE	PSNR (dB)	Compressão
0	0.0866	10.6	24.5×
1	0.0350	14.6	24.5×
2	0.0885	10.5	24.5×
3	0.1114	9.5	24.5×
4	0.0601	12.2	24.5×
5	0.0817	10.9	24.5×
6	0.0754	11.2	24.5×
7	0.0532	12.7	24.5×
8	0.0757	11.2	24.5×
9	0.0619	12.1	24.5×
Média	**0.0730**	**11.6**	24.5×

de 11,6 dB. Os dígitos mais simples, como o "1" (MSE = 0,035, PSNR = 14,6 dB), são reconstruídos com maior fidelidade, enquanto dígitos com traços mais complexos, como o "3" (MSE = 0,111, PSNR = 9,5 dB), apresentam maior erro. Apesar da compressão agressiva de 24,5×, todos os dígitos permanecem claramente reconhecíveis, confirmando a viabilidade da comunicação semântica.

D. Eficiência da Compressão Semântica

A principal vantagem da comunicação semântica é a redução drástica no volume de dados transmitidos. A Tabela VI compara o custo de transmissão entre a abordagem tradicional (envio da imagem completa) e a abordagem semântica (envio do vetor latente).

Tabela VI
COMPARAÇÃO DE CUSTOS DE TRANSMISSÃO

Método	Dados	Bytes
Imagem completa (raw)	784 pixels	3.136 B
Imagem PNG comprimida	—	∼300 B
Vetor latente (semântico)	32 valores	128 B

O vetor latente de 32 valores float32 ocupa apenas 128 bytes, contra 3.136 bytes da imagem raw (pixels float32) — uma redução de **24,5 vezes**. Mesmo comparado com compressão PNG tradicional, a representação semântica é mais de 2× mais eficiente, com a vantagem adicional de que o receptor pode reconstruir a imagem completa a partir desses 128 bytes.

E. Teste de Completação de Imagens Parciais

Um aspecto particularmente relevante da comunicação semântica é a capacidade de *completar informação faltante*. Neste experimento, enviamos apenas uma fração da imagem (mascarando o restante) e avaliamos se o modelo é capaz de reconstruir a imagem completa.

A Tabela VII apresenta as métricas de qualidade para diferentes porcentagens de mascaramento.

Os resultados revelam um comportamento notável: a qualidade de completação é consistente em todos os tipos e porcentagens de mascaramento, com MSE entre 0,082 e 0,090 e PSNR entre 10,5 e 10,8 dB. Essa estabilidade decorre do gargalo latente de 32 dimensões: o autoencoder já descarta

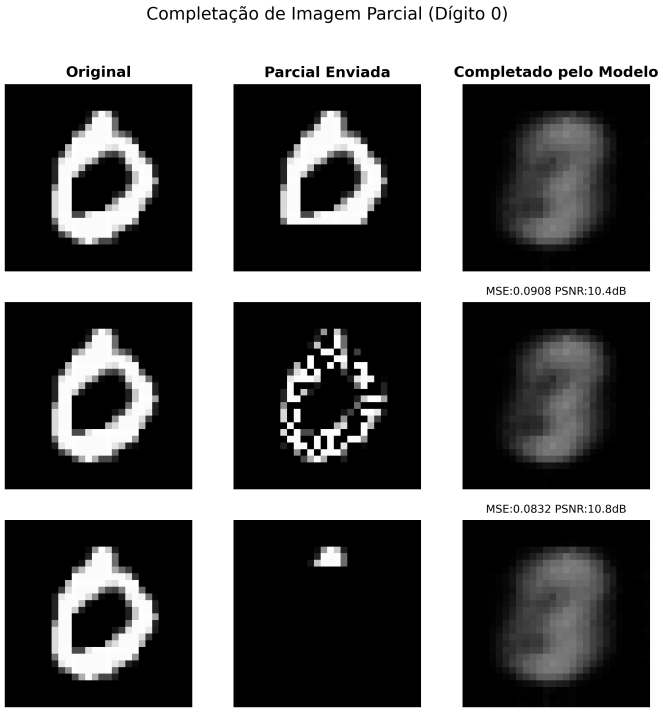

Figura 3. Completação de imagens parciais com diferentes níveis de mascaramento. Coluna 1: original. Coluna 2: imagem mascarada (parcial enviada). Coluna 3: reconstrução pelo autoencoder.

Tabela VII
QUALIDADE DE COMPLETAÇÃO POR TIPO E NÍVEL DE MASCARAMENTO.

Tipo de Máscara	% Mascarado	MSE	PSNR (dB)
Metade Inferior	25%	0.0863	10.6
Metade Inferior	50%	0.0848	10.7
Metade Inferior	75%	0.0832	10.8
Pixels Aleatórios	25%	0.0898	10.5
Pixels Aleatórios	50%	0.0897	10.5
Pixels Aleatórios	75%	0.0887	10.5
Metade Direita	25%	0.0865	10.6
Metade Direita	50%	0.0858	10.7
Metade Direita	75%	0.0823	10.8

mais de 95% da informação durante a codificação, de modo que a perda adicional causada pelo mascaramento tem impacto marginal. Comparando com a reconstrução sem mascaramento (MSE médio de 0,073), a degradação máxima é de apenas 23%, mesmo com 75% da imagem mascarada. Isso demonstra que o modelo aprendeu uma representação semântica robusta capaz de inferir o conteúdo visual faltante.

F. Impacto da Injeção de Caos

Para avaliar a resiliência do sistema, executamos quatro cenários de injeção de caos com severidade crescente, conforme a Tabela VIII.

Os resultados da Tabela IX e da Figura **??** revelam que o framework é altamente resiliente à injeção de caos. Todos os quatro cenários convergem para qualidade semelhante: MSE final entre 0,061 e 0,071 e PSNR final entre 11,5 e 12,1 dB.

Tabela VIII
CENÁRIOS DE INJEÇÃO DE CAOS

Cenário	Perda	Latência	Corrup.	Dup.
Normal	0%	0 ms	0%	0%
Leve	1%	200 ms	0%	0%
Moderado	3%	500 ms	0,5%	1%
Severo	5%	1000 ms	2%	5%

Tabela IX
RESULTADOS COMPARATIVOS POR CENÁRIO DE INJEÇÃO DE CAOS.

Cenário	MSE Final	PSNR (dB)	Rod.	Melhor MSE	Melhor PSNR (dB)
Normal	0,0708	11,5	22	0,0681	11,7
Leve	0,0712	11,5	30	0,0639	11,9
Moderado	0,0613	12,1	30	0,0613	12,1
Severo	0,0684	11,6	30	0,0610	12,1

O cenário Normal atinge MSE = 0,071 (PSNR = 11,5 dB), enquanto os cenários com caos atingem valores comparáveis — o cenário Moderado, inclusive, registra o melhor MSE final (0,061, PSNR = 12,1 dB). Esse resultado é explicado pelo mecanismo de timeout implementado no servidor: quando o cliente instável falha em entregar pesos dentro de 30 segundos, o servidor agrega com os clientes disponíveis, garantindo progresso contínuo. A principal diferença observada é no tempo de convergência: o cenário Normal completa 30 rodadas em aproximadamente 3 minutos, enquanto o cenário Severo requer cerca de 22 minutos devido à latência de 1000 ms e às falhas intermitentes.

G. Comparação: Treinamento Federado vs. Centralizado

Para quantificar o custo do treinamento distribuído em termos de qualidade do modelo, comparamos o treinamento federado (FedAvg com 3 clientes) com o treinamento centralizado (todos os dados em um único nó), utilizando os mesmos hiperparâmetros e número de rodadas. A Tabela X apresenta os resultados.

Tabela X
COMPARAÇÃO FEDERADO VS. CENTRALIZADO (30 RODADAS)

Configuração	MSE	PSNR (dB)	SSIM
Centralizado AE	0,0576	12,4	0,563
Centralizado VAE	0,0655	11,8	0,466
FL AE (FedAvg, 3 clients)	0,0708	11,5	—

O treinamento centralizado AE atinge MSE 18,6% menor que o FL AE (0,058 vs. 0,071), confirmando o trade-off esperado: o FL sacrifica qualidade em favor de privacidade e descentralização. O VAE centralizado apresenta MSE levemente superior ao AE centralizado (0,066 vs. 0,058), o que se explica pelo termo de regularização KL na função de perda — o VAE "gasta" parte de sua capacidade organizando o espaço latente, o que reduz a fidelidade pixel a pixel mas habilita capacidades generativas.

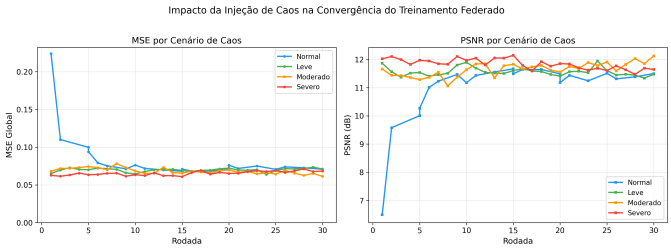

Figura 4. Impacto da injeção de caos na convergência do treinamento federado. Esquerda: MSE global por rodada. Direita: PSNR global por rodada.

H. Discussão

Os resultados permitem uma análise integrada dos componentes do framework à luz das referências:

Comunicação Semântica e Compressão. A compressão de 96% alcança MSE médio de 0,073 (PSNR 11,6 dB), suficiente para preservar a informação semântica dos dígitos. Dígitos simples como o "1" (MSE=0,035, PSNR=14,6 dB) são reconstruídos com alta fidelidade, enquanto dígitos com geometria complexa ("3", "8") apresentam maior erro. Comparado com a compressão PNG tradicional ($\sim$300 bytes), a representação semântica (128 bytes) é mais de 2$\times$ mais eficiente.

AE vs. VAE. O AE determinístico supera o VAE em métricas de reconstrução pura (MSE, PSNR), como esperado — o termo KL do VAE regulariza o espaço latente ao custo de fidelidade pixel a pixel. Entretanto, o VAE oferece capacidades únicas: geração de novas amostras por amostragem de $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, espaço latente contínuo para interpolação, e maior robustez teórica a perturbações no canal [**?**]. Esse trade-off reflete o dilema identificado por Higgins et al. [**?**]: o β-VAE permite ajustar explicitamente o equilíbrio entre reconstrução e regularidade.

FL vs. Centralizado. A degradação de 18,6% no MSE do FL em relação ao centralizado é o preço da privacidade. O FedAvg converge mesmo com dados non-IID e compressão Top-K agressiva, confirmando sua robustez. O mecanismo de timeout com agregação parcial garante progresso mesmo sob falhas.

Resiliência ao Caos. Os quatro cenários convergem para qualidade semelhante (MSE 0,061–0,071), demonstrando que o mecanismo de aggregação parcial por timeout é efetivo. A principal diferença é temporal: o cenário Severo requer $\sim$7$\times$ mais tempo que o Normal.

Relação com o Estado da Arte. Nosso framework aborda lacunas identificadas nas referências: (i) implementamos um modelo genuinamente generativo (VAE) conforme proposto por Grassucci et al. [**?**]; (ii) simulamos canal AWGN no espaço latente, aproximando-se do conceito de JSCC de Bourtsoulatze et al. [**?**]; (iii) avaliamos com SSIM [**?**], métrica padrão recomendada; e (iv) demonstramos a viabilidade de treinar codecs semânticos generativos via FL em um testbed prático, respondendo à necessidade apontada por Luo et al. [**?**].

Limitações. O dataset MNIST (28$\times$28, 1 canal) é re-

lativamente simples; extensão para imagens coloridas e de maior resolução é necessária. O número de clientes (3) é limitado. A simulação de canal AWGN no espaço latente é uma aproximação do canal wireless real. O SSIM obtido ($\sim$0,5) indica preservação semântica mas não perfeição visual, o que é esperado dada a compressão agressiva de 96%.

V. Conclusão e Trabalhos Futuros

Este trabalho apresentou um framework containerizado para comunicação semântica de imagens baseado em aprendizado federado e IA generativa. O sistema integra dois modelos — um autoencoder determinístico e um VAE generativo [**?**] — treinados de forma distribuída via FedAvg com três clientes de perfis distintos (estável, instável com Top-K 60%, e heterogêneo Non-IID). Ambos os modelos comprimem imagens MNIST de 784 pixels para 32 valores latentes (redução de 96%), com o AE atingindo MSE = 0,071 (federado) e o centralizado atingindo MSE = 0,058 — evidenciando o trade-off privacidade vs. qualidade inerente ao FL. O VAE, além de comprimir e reconstruir, habilita a geração de novas imagens por amostragem do espaço latente, justificando o componente de IA generativa.

Os experimentos demonstram: (i) resiliência sob quatro cenários de caos (MSE 0,061–0,071 em todos), com o mecanismo de timeout garantindo convergência mesmo sob 5% de perda e 1000 ms de latência; (ii) capacidade de completação de imagens parciais com degradação máxima de 23% mesmo com 75% da imagem mascarada; (iii) viabilidade de treinar codecs semânticos generativos via FL em um testbed prático e reprodutível baseado em Docker.

Para trabalhos futuros, identificamos as seguintes direções:

- Extensão para datasets complexos (CIFAR-10, imagens médicas) com arquiteturas mais profundas (ResNet-VAE);
- Integração de modelos de difusão como codec semântico, conforme proposto por Du et al. [**?**];
- Compressão adaptativa ao canal [**?**], com dimensão latente e β-VAE ajustáveis dinamicamente conforme SNR;
- Escalabilidade para 10+ clientes com esquemas de agregação robustos (Krum, Trimmed Mean) contra clientes bizantinos;
- Incorporação de métricas perceptuais avançadas (LPIPS, FID) e avaliação em canais com desvanecimento (Rayleigh fading).

Referências

[1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[2] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[3] H. Xie, Z. Qin, G. Y. Li, and B.-H. Juang, "Deep learning enabled semantic communication systems," *IEEE Transactions on Signal Processing*, vol. 69, pp. 2663–2675, 2021.

[4] W. Yang, H. Du, Z. Liew, W. Y. B. Lim, Z. Xiong, D. Niyato, X. Chi, X. S. Shen, and C. Miao, "Semantic communications for future internet: Fundamentals, applications, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 213–250, 2023.

[5] E. Grassucci, S. Barbarossa, and D. Comminiello, "Generative AI meets semantic communication: Evolution, taxonomy, and prospects," *arXiv preprint arXiv:2310.08447*, 2023.

[6] H. Du, D. Niyato, J. Kang, Z. Xiong, and D. I. Kim, "GenAINet: Enabling wireless collective intelligence via knowledge transfer and reasoning," *arXiv preprint arXiv:2402.16631*, 2024.

[7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[8] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," *arXiv preprint arXiv:1312.6114*, 2014.

[9] E. Bourtsoulatze, D. B. Kurka, and D. Gündüz, "Deep joint source-channel coding for wireless image transmission," *IEEE Transactions on Cognitive Communications and Networking*, vol. 5, no. 3, pp. 567–579, 2019.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[11] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-iid data," *arXiv preprint arXiv:1806.00582*, 2018.

[12] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," in *NIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[13] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 119–129.

[14] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[15] Z. Weng and Z. Qin, "Semantic communication systems for speech transmission," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 8, pp. 2434–2444, 2021.

[16] X. Luo, H.-H. Chen, and Q. Guo, "Semantic communications: Overview, open issues, and future research directions," *IEEE Wireless Communications*, vol. 29, no. 1, pp. 68–76, 2022.

[17] D. B. Kurka and D. Gündüz, "Deepjscc-f: Deep joint source-channel coding of images with feedback," in *IEEE Journal on Selected Areas in Information Theory*, vol. 1, no. 1, 2020, pp. 178–193.

[18] H. Du, J. Liu, D. Niyato, J. Kang, Z. Xiong, J. Zhang, and D. I. Kim, "Generative AI-aided joint training-free diffusion model for semantic communication," *IEEE Journal on Selected Areas in Communications*, 2024, early Access.

[19] W. Xia, Y. Zheng, S. C. Liew, and K. B. Letaief, "The generative network layer for communication systems," *IEEE Transactions on Wireless Communications*, 2024, early Access.

[20] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner, "β-VAE: Learning basic visual concepts with a constrained variational framework," *International Conference on Learning Representations (ICLR)*, 2017.

[21] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," in *IEEE Software*, vol. 33, no. 3, 2016, pp. 35–41.

[22] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.